

CSA0613 DESIGN ANALYSIS OF ALGORITHMS

Name: Nivasini V.M

Reg number: 192511356

CO2_ASSESSMENT TOOL 1_APPLICATION_BASED QUESTIONS

Q26. Binary Search – Last Element Analysis

Given the sorted dataset [2, 6, 10, 14, 18, 22, 26, 30], apply Binary Search to find the element 30. Clearly interpret the problem and justify the use of Binary Search.

Show each

step involved in dividing the search space until the element is found. Count the total number of comparisons performed. Analyze the time complexity of the approach.

Verify

the correctness of the result and interpret its efficiency when the target is at the last position.

Aim:

To implement and execute the given algorithm/program using Python, analyze its performance, and upload the source code, report, and supporting files in a structured GitHub repository.

Algorithm

1. Start.
2. Read the sorted array and the target element.

3. Initialize $low = 0$ and $high = n - 1$.
4. Repeat while $low \leq high$:
 - Calculate $mid = (low + high) // 2$.
 - If $array[mid] == target$, display the position of the element and stop.
 - If $target < array[mid]$, set $high = mid - 1$.
 - Otherwise, set $low = mid + 1$.
5. If the loop ends without finding the element, display **“Element not found”**.
6. Stop.

Python Program

```
1 # Binary Search - Last Element Analysis
2 arr = [2, 6, 10, 14, 18, 22, 26, 30]
3 target = 30
4 low = 0
5 high = len(arr) - 1
6 comparisons = 0
7 while low <= high:
8     mid = (low + high) // 2
9     comparisons += 1
10    if arr[mid] == target:
11        print("Element found at index:", mid)
12        print("Total comparisons:", comparisons)
13        break
14    elif target < arr[mid]:
15        high = mid - 1
16    else:
17        low = mid + 1
18 else:
19    print("Element not found")
```

Output

Output

```
Element found at index: 7
```

```
Total comparisons: 4
```

```
=== Code Execution Successful ===
```

Result

The Binary Search program was successfully implemented and executed. The target element 30 was found at index 7 after 4 comparisons. The algorithm correctly searched the sorted array by repeatedly dividing the search space into half, demonstrating an efficient $O(\log n)$ time complexity.